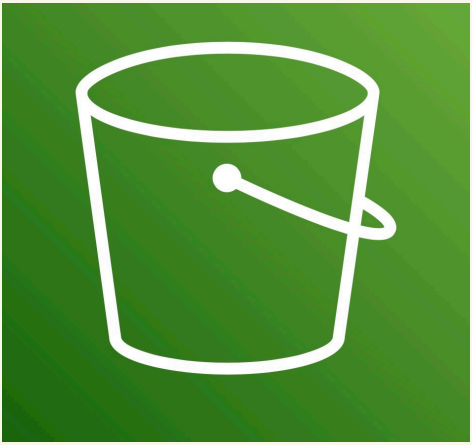
Everything you need to know about -

S3 MULTIPART UPLOAD USING AWS CLI



AWS S3 multipart upload using AWS CLI

S3 → massive, infinitely scalable Google drive for developers

you → files → S3 bucket

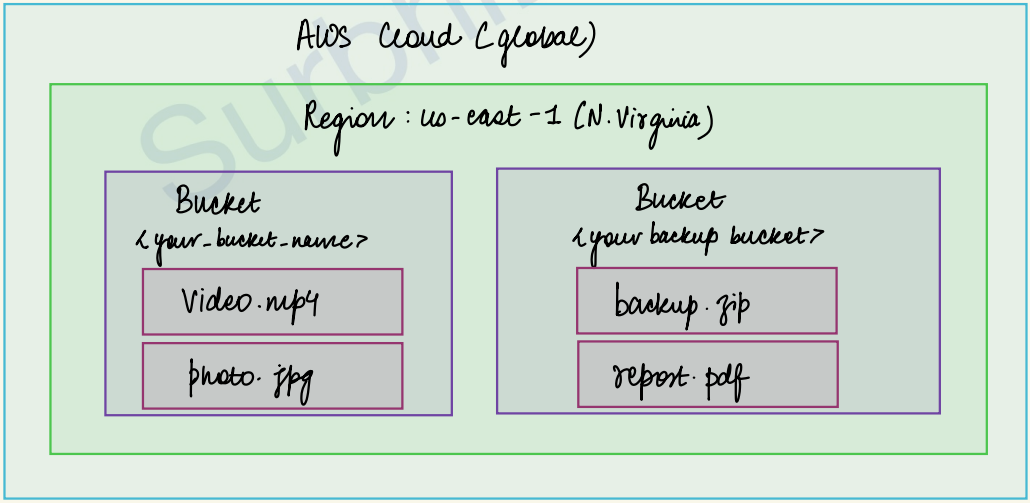
handles → hard drives, servers, file systems } you dont have to think about them

your files are called as objects

and they live inside buckets

The key difference from a regular hard drive → S3 = object storage ✓
↳ file storage ✗

Every object gets a → unique ID, metadata, URL } → you can access it over web (HTTP)
↓
not like a USB drive ⊗



- bucket names are globally unique → like domain names
- bucket are region specific → but their names must be unique globally
- Objects inside buckets can be upto 5TB in size
- you access everything over REST API's (HTTP GET, PUT, DELETE)

SurabhiExplains

What is Multipart upload?

your laptop

shaky internet

↓

you upload 1GB video

↓

halfway through → your connection drops

→ If you are doing Normal upload

↓

you start from 0

↓

Painful!!

⇓

Multipart upload solves this problem

↓

break the file into chunks → and upload each one independently →

if chunk 3 fails → you re-upload chunk 3

Phase 1 - Initiate

Send create-multipart upload → AWS returns a unique upload ID

↓

Phase 2 - upload parts

(use upload ID for each)

Part 1 - Xaa	Part 2 - Xab	Part 3 - Xac	Part - Xad
40 MB → ETag	40 MB → ETag	40 MB → ETag	23 MB → ETag

each upload returns an ETag (a fingerprint/checksum for that part)

Save all ETags → you need them in phase 3

↓

Phase 3 - Complete

Send list.json (all part Numbers + ETags) → S3 assembles final object

SurbhiExplains

ETags ⇒ it's an MD5 checksum → fingerprint proving the data arrived intact

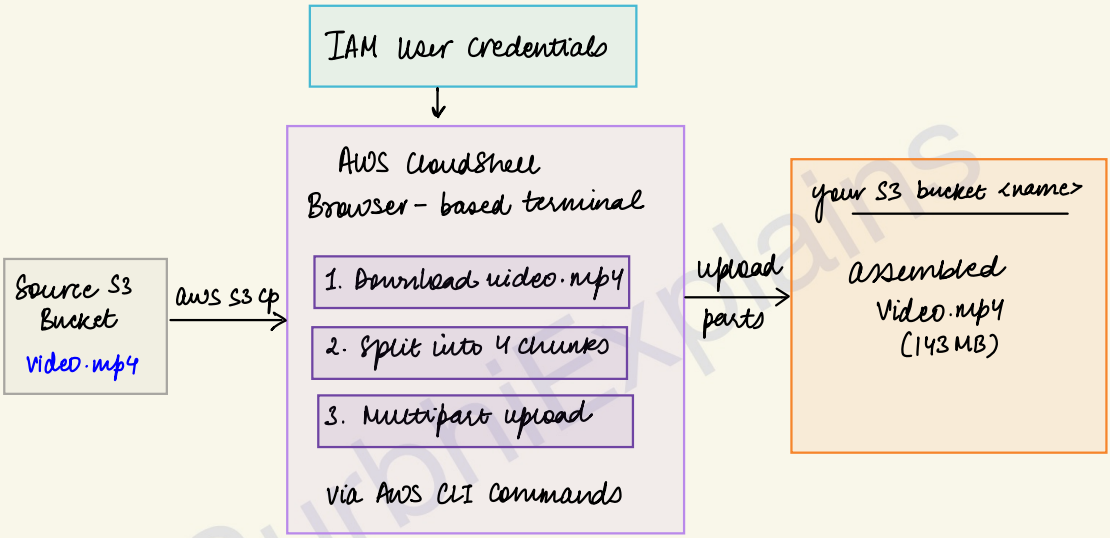
↓

when you complete the upload, S3 uses ETags to verify each part is accounted for and uncorrupted

↓

It then stitches them in order by part number

lab Architecture



lab Steps

Step 1 → Sign in AWS console → Create an S3 bucket

↳ ACL's disabled

↓

mean access is controlled by IAM policies → not per-object access lists

Step 2 → CloudShell = Browser based Linux terminal that runs inside AWS itself

↳ It has AWS CLI pre installed and already authenticated with your credentials

1.) Copy video file →

```
aws s3 cp s3://<source_bucket_name>/video.mp4 /home/cloudshell-user/
```

SurbhiExplains

This pulls a pre-existing 143 MB video from a source bucket into your cloudshell's local disk

gives you 1 GB of persistent storage

2.) View the file → `ls -l`

-l flag shows file size, permissions & time stamps in long format

you'll see file is 143MB → above 100MB threshold where AWS recommends switching to multipart

Changes according to your file

3.) Split the file → `split -b 40M video.mp4`

linux split command divides the file into 40MB chunks

(143 MB ÷ 40 MB = 3.575) → you get 4 files

- xaa (40MB)
- xab (40MB)
- xac (40MB)
- xad (23MB)

The x prefix and alphabetical suffix (aa, ab ...) are default naming convention of the split command

4.) Initiate multipart upload →

```
aws s3api create-multipart-upload --bucket <bucket-name> --key video.mp4
```

AWS responds with a long uploadID string.

This ID ties all your parts together → if you loose it you cant complete the upload

--key is object name in S3

its like a filename (though it can contain / to simulate folder paths)

SurbhiExplains

5.) Upload each part →

```
aws s3api upload-part --bucket [bucket-name] --key video.mp4 \  --part-number 1 --body xaa --upload-id [your-uploadID]
```

Change --part-number and --body each time
↓
repeat it 4 times (the number of chunks your file is divided into)

Each response gives you ETag
↓
Save every single one !!

⇒ --part-number determines the assembly order not the filename order

6.) Create list.json → This is the "manifest" file. It tells S3 exactly which ETags correspond to which part numbers

```
{
  "Parts": [
    {
      "PartNumber": 1,
      "ETag": "\"70418ed5e552ea21deb8785359e69e28\""
    },
    {
      "PartNumber": 2,
      "ETag": "\"e0c16ead703bfcd1b36b339d9ae1901d\""
    },
    {
      "PartNumber": 3,
      "ETag": "\"56734bc19b453aab5144de4454945609\""
    },
    {
      "PartNumber": 4,
      "ETag": "\"7786233d68592caf07e93521cbd0a80e\""
    }
  ]
}
```

escaped quotes inside ETag values are because ETags are returned with literal quote characters inside them

Nano text editor
↓
Ctrl+X and then Y
↓
then Enter
↓
Save and exits it

7.) Complete the multipart upload

```
aws s3api complete-multipart-upload \
  --multipart-upload file://list.json \
  --bucket [bucket_name] --key video.mp4 \
  --upload-id [YOUR_UPLOAD_ID]
```

file:// is a special prefix that tells CLI to read JSON from a local file

↓
S3 now assembles all 4 parts in order & creates a single 143 MB video.mp4 object.

SurbhiExplains

The parts now cease to exist individually

Step 3 → Verify in console → you navigate to S3 in console and confirm video.mp4 exists in your bucket.

Some beyond the lab content

① 5MB minimum part because → it prevents from people from using multipart to upload 1000's of tiny 1KB parts
↓
This would be inefficient for S3's backend

② Maximum parts = 10,000 parts per multipart upload. So theoretically a single object can be 10,000 x 5GB = 50TB
↓
The actual S3 Object size limit

③ Abandoned uploads cost money
↓
you initiate a multipart upload → never complete it / abort it
↓
The partial parts sit in S3 & you get charged for storage
↓
Always abort failed uploads with `aws s3api abort-multipart-upload`
OR
We can set a lifecycle policy to auto-delete incomplete multipart upload after N days

④ Parallel uploading → you can upload all 4 parts simultaneously in parallel → concurrently across multiple threads, dramatically improving speed on fast networks

⑤ ETags and MD5 → for single-part objects → ETag is exactly the MD5 hash of data. For multipart objects its `MD5(concatenated_part_MD5s)-N`
↓
Important for identity verification

Suffix N means → N = number of parts

SurbhiExplains